

Basic MPC 原理与代码详解

单水箱连续流量约束 MPC：从数学原理到 MATLAB 实现

适用代码目录

D:/文档/墨大课程/capstone/EMPC_Water_Electricity_Forecast/CKH_EMPC/basic_mpc

第一部分：模型原理与运行逻辑
第二部分：五步阅读法与逐段代码解释

阅读说明

这份讲义刻意把“原理”和“代码细节”分开。第一部分先回答系统为什么这样建模、MPC每小时到底做什么；第二部分再把数学对象逐个对应到代码。

建议第一次阅读时先完整读完第一部分，不要急着记矩阵。第二部分阅读到控制器文件时，再回看状态方程、目标函数与约束。

全文符号

符号	含义	单位/范围
$x(k)$	第 k 个小时开始时的水箱水位	m
$d(k)$	用户从水箱取走的用水流量	m^3/s
$u(k)$	MPC给出的归一化期望总流量	0 到 1
q_{max}	泵站可要求的最大总流量	$0.05 \text{ m}^3/\text{s}$
$q_{\text{request}}(k)$	MPC要求流量， $q_{\text{max}} u(k)$	m^3/s
$q_{\text{actual}}(k)$	Decision模块实际实现的流量	m^3/s
$r(k)$	目标水位	m
N	预测时域长度	24步，即24小时

一个必须先守住的边界

这版模型是基础约束水箱 MPC，不是完整的经济模型预测控制 EMPC。它没有把电价、功率、启停成本或二进制泵状态放入目标函数。price 字段目前只是预留。

第一部分 模型原理与运行逻辑

这版 basic_mpc 的本质

单水箱、连续流量、带约束、滚动优化的基础 MPC。

它现在解决的不是电价问题，也不直接决定具体哪台水泵开关，而是在每个小时回答：未来24小时，泵站每小时应该提供多大比例的总流量，才能让水位安全、接近目标，而且不要突然大幅改变？

1. MPC的直觉：不断重做计划的水库管理员

- 测量当前水位 $x(k)$ 。
- 取得未来24小时的预测用水量 and 目标水位。
- 尝试不同的未来供水方案。
- 用水箱模型计算每套方案对应的未来水位。
- 找出代价最小、又不违反约束的24小时方案。
- 只执行第一个小时；一小时后重新测量并重新规划。

假设当前算出的最优计划为：

$$U^* = [0.35, 0.50, 0.55, \dots]$$

当前只执行 $u(0)=0.35$ 。到了下一小时，0.50不会被机械照搬，而是用新水位和新预测重新求解。这就是滚动时域控制。

2. 模型变量

水位 $x(k)$

$x(k)$ 表示第 k 个小时开始时的水箱水位。当前设置为 $1.40 \leq x(k) \leq 3.37$ m，初始水位 $x(0)=2.30$ m。

控制量 $u(k)$

$u(k)$ 是归一化的期望总流量，不是水泵开关。 $u=0$ 表示零流量， $u=1$ 表示最大流量。

$$q_{\text{request}}(k) = q_{\text{max}} u(k), q_{\text{max}} = 0.05 \text{ m}^3/\text{s}$$

u	要求流量 q_{request}
0	0 m ³ /s
0.4	0.020 m ³ /s
0.7	0.035 m ³ /s
1	0.050 m ³ /s

用水需求 $d(k)$

$d(k)$ 表示用户从水箱取走的水流量，单位为 m³/s。这里的 demand 是用水需求，不是AEMO数据里的电力系统 TOTALDEMAND。

3. 状态方程来自质量守恒

水箱满足最直接的守恒关系：储水量变化 = 流入水量 - 流出水量。若横截面积为 A 、水位为 x ，则体积 $V=Ax$ 。

$$A[x(k+1)-x(k)] = \Delta t [q_{\text{in}}(k)-d(k)]$$

$$x(k+1) = x(k) + (\Delta t/A)[q_{\text{in}}(k)-d(k)]$$

基础MPC令 $q_{\text{in}}(k)=q_{\text{max}} u(k)$ ，因此：

$$x(k+1) = x(k) + (\Delta t/A)[q_{\text{max}} u(k)-d(k)]$$

这就是整个基础MPC最核心的物理模型。它是线性的，因此后面能整理成二次规划。

4. 代入当前数值

当前 $\Delta t=3600$ s, $A=1000$ m², $q_{\text{max}}=0.05$ m³/s, 所以 $\Delta t/A=3.6$, $(\Delta t/A)q_{\text{max}}=0.18$ 。

$$x(k+1) = x(k) + 0.18u(k) - 3.6d(k)$$

若没有用水且满流量运行一小时，水位上升0.18 m。若需求 $d=0.025 \text{ m}^3/\text{s}$ ，为维持水位不变，需要 $q_{\max} u=d$ ，因此 $u=0.5$ 。

如果只给 $u=0.4$ ，则 $\Delta x=3.6(0.05 \times 0.4 - 0.025) = -0.018 \text{ m}$ ，即一小时下降1.8 cm。

5. 为什么预测24步

只看当前误差的控制器通常是“水位低了再补水”。MPC知道未来需求，因此即使当前水位正常，也可以在高需求到来前提前储水。

$$x(i|k) = x(k) + (\Delta t/A) \sum_{j=0 \dots i-1} \{q_{\max} u(j|k) - d(j|k)\}$$

$x(i|k)$ 表示在当前时刻 k 对未来第 i 步水位的预测。控制器一次求解24个控制变量 $U=[u(0|k), \dots, u(23|k)]$ 。

6. MPC的目标函数：三张罚单

$$J = \sum w_x [x(i|k) - r(i|k)]^2 + \sum w_u u(i|k)^2 + \sum w_{\Delta u} [u(i|k) - u(i-1|k)]^2$$

第一张：水位偏离目标

$J_x = w_x \sum (x-r)^2$ 。正负误差都会受到惩罚，而且大误差增长更快。例如误差从0.1 m增至0.2 m，平方惩罚从0.01增至0.04。

第二张：控制输入太大

$J_u = w_u \sum u^2$ 。它阻止控制器无意义地长期要求大流量，但它不是真正的电费，因为当前没有电价和功率模型。

第三张：控制变化太剧烈

$J_{\Delta u} = w_{\Delta u} \sum (\Delta u)^2$ 。它让控制指令更平滑，避免0、1、0、1式跳变。当前权重为 $w_x=50$ 、 $w_u=0.02$ 、 $w_{\Delta u}=0.50$ 。三个项尺度和单位不同，不能把权重数值直接解释成重要性倍数。

7. 硬约束与软目标

$x \approx r$ 是软目标：偏离参考水位只会增加代价。以下是硬约束：候选方案只要违反，就不能成为可行解。

$$1.40 \leq x(i|k) \leq 3.37$$

$$0 \leq u(i|k) \leq 1$$

$$|u(i|k) - u(i-1|k)| \leq 0.35$$

因此水位可以暂时不等于2.30 m，但预测水位不能穿过1.40 m或3.37 m。

8. 一个变化约束的数值例子

初始上一时刻输入 $u(-1)=0$ 。因为 $|u(0)-u(-1)| \leq 0.35$ ，所以第一小时最多给 $u(0)=0.35$ 。若需求恒为 $0.025 \text{ m}^3/\text{s}$ ，维持水位需要 $u=0.5$ ，但第一步受变化约束限制。

$$\Delta x = 3.6(0.05 \times 0.35 - 0.025) = -0.027 \text{ m}$$

第一小时水位会下降约2.7 cm。第二小时可从0.35增加到最多0.70，因此能够逐渐调整到约0.5。

9. 从预测模型变成quadprog

用下三角累加矩阵 L 可以把24条递推式一次写完：

$$X = X_{\text{base}} + \Phi U$$

$$X_{\text{base}} = x(k)1 - (\Delta t/A)L D, \Phi = (\Delta t/A)q_{\max} L$$

X_{base} 表示假设未来水泵不供水、只受用水需求影响时的水位轨迹； ΦU 表示控制输入给水位带来的累计提升。

再用差分矩阵 D 写出 $\Delta U = DU - p$ ，目标函数便能整理为标准二次规划：

$$\min \frac{1}{2} U^T H U + f^T U, \text{ s.t. } AU \leq b, lb \leq U \leq ub$$

quadprog并不理解水箱或水泵；它只接收 H 、 f 、 A 、 b 和变量上下界。控制器代码负责把物理问题翻译成这些矩阵。

10. 一轮完整运行

- 取得未来24小时的需求与目标水位窗口。
- 求解24小时控制计划和预测水位。
- 只把第一个连续流量要求发送给Decision模块。
- Decision模块返回实际流量；当前模拟器假设实际值完全等于要求值。
- 水箱只用实际流量更新下一小时水位。
- 窗口向前移动一小时，再次预测、优化、执行。

测量 → 预测 → 优化24步 → 执行1步 → 更新水箱 → 重新优化

11. 当前模型边界

已经实现	尚未实现
单水箱动态模型	电价成本与价格预测
24小时预测与滚动闭环	泵功率曲线
水位、流量、变化率约束	具体泵组开关与二进制变量
连续流量二次规划	启停成本、最小运行时间
MPC与Decision接口	真实执行偏差与预测不确定性

准确称呼是“基础约束水箱MPC”或“连续流量跟踪MPC”，不能直接称为完整EMPC。

第二部分 五步阅读代码

阅读顺序不是按文件夹字母顺序，而是按控制系统理解顺序：先知道参数，再看输入数据，再看闭环，再进入优化器，最后看模块接口。

步骤	文件	核心问题
1	basic_mpc_config.m	系统有哪些参数和约束？
2	make_mock_decision_data.m	控制器收到什么未来数据？
3	simulate_basic_mpc.m	每小时的闭环怎样运行？
4	constrained_tank_mpc.m	数学问题怎样交给quadprog？
5	三个interfaces文件	MPC和Decision怎样交换数据？

步骤一 basic_mpc_config.m：先认清参数

```
function config = basic_mpc_config()
%BASIC_MPC_CONFIG 基础水箱 MPC 的统一参数。
% MPC 只计算连续的期望总流量；具体水泵组合由 Decision 模块负责。
```

函数不需要输入，返回结构体config。把参数集中在一个文件里，可以避免控制器、仿真器和接口各自保存不同数值。开头注释明确了边界：MPC输出连续总流量要求，具体泵组合属于Decision模块。

1.1 物理模型参数

```
config.dt_seconds = 3600;
config.tank_area_m2 = 1000;
config.max_requested_flow_m3s = 0.050;
```

参数	值	含义
dt_seconds	3600 s	每个控制步为1小时
tank_area_m2	1000 m²	水箱横截面积
max_requested_flow_m3s	0.05 m³/s	u=1时的最大要求流量

这三个参数共同决定u对水位的作用强度： $(\Delta t/A)q_{\max}=0.18\text{ m/步}$ 。

1.2 水位边界与初值

```
config.level_min_m = 1.40;
config.level_max_m = 3.37;
config.level_initial_m = 2.30;
```

1.40 m和3.37 m进入优化约束；2.30 m只用于仿真开始时设置 $x(0)$ ，并不强迫未来水位始终等于2.30 m。目标水位由输入数据里的level_ref给出。

1.3 预测时域与控制范围

```
config.horizon_steps = 24;
config.u_min = 0.0;
config.u_max = 1.0;
config.du_max = 0.35;
```

$N=24$ 表示每次规划未来24个一小时步。u被限制在0到1；相邻小时控制变化最多0.35。若上一小时 $u=0.20$ ，则下一小时数学允许范围是 $[-0.15,0.55]$ ，再与 $[0,1]$ 求交，得到 $[0,0.55]$ 。

1.4 目标函数权重

```
config.weight_level = 50.0;
config.weight_input = 0.02;
config.weight_move = 0.50;
```

weight_level惩罚水位跟踪误差，weight_input惩罚持续使用较大输入，weight_move惩罚输入变化。调权重实际上是在调三种偏好之间的折中。

提高哪个权重	通常会出现的趋势
weight_level	更努力贴近参考水位，可能使用更积极的流量变化
weight_input	倾向于较小u，水位跟踪可能变差
weight_move	控制更平滑，但响应可能变慢

这些只是趋势，不是绝对结果；最终行为还取决于约束、需求和数值尺度。

步骤二 make_mock_decision_data.m: 理解输入数据

这个文件生成仿真输入。它不是预测算法，也不是Decision优化器；它只是假装外部模块已经提供了未来需求、目标水位和时间。

2.1 默认输入

```
if nargin < 1 || isempty(start_time)
    start_time = datetime(2026, 2, 1, 0, 0, 0);
end
if nargin < 2 || isempty(n_steps)
    n_steps = 72;
end
if nargin < 3 || isempty(mode)
    mode = 'periodic';
end
```

nargin是实际传入的参数数量；isempty检查某个参数是否传了空值[]。因此不传参数时，从2026-02-01 00:00开始，生成72步周期数据。

2.2 时间轴

```
k = (0:n_steps-1);
decision_output.time = start_time + hours(k);
```

k是从0开始的列向量。n_steps=72时，k=[0,1,...,71]；加到start_time后形成每小时一个时间戳。第0步就是开始时间，第24步是次日同一时刻。

2.3 fixed模式

```
case 'fixed'
    decision_output.demand = 0.025 * ones(n_steps, 1);
    decision_output.level_ref = 2.30 * ones(n_steps, 1);
```

所有时刻的需求都为0.025 m³/s，参考水位都为2.30 m。这个场景适合检查稳态：忽略变化约束影响后，平衡输入应接近 $u=d/q_{\max}=0.5$ 。

2.4 periodic模式

```
daily_phase = 2*pi*k/24;
decision_output.demand = 0.025 ...
    + 0.010*sin(daily_phase - pi/2);
decision_output.level_ref = 2.30 ...
    + 0.15*sin(daily_phase);
```

$$\theta(k)=2\pi k/24$$

θ 每24步增加 2π ，因此需求和参考水位每24小时重复。需求均值0.025、振幅0.010，范围0.015到0.035 m³/s；参考水位均值2.30、振幅0.15，范围2.15到2.45 m。

k	需求 d(k)	参考水位 r(k)
0	0.015	2.30
6	0.025	2.45
12	0.035	2.30
18	0.025	2.15
24	0.015	2.30

参考水位在第6小时达到高峰，需求在第12小时达到高峰。这个人为相位差会引导控制器在需求峰值前储水，用于展示MPC的前瞻性；它不是由真实数据论证出来的运行策略。

2.5 非法模式、price、source和版本号


```
otherwise
    error('make_mock_decision_data:Mode', ...
        'mode 必须是 "fixed" 或 "periodic"。');
end

decision_output.price = nan(n_steps, 1);
decision_output.source = ['mock_', lower(mode)];
decision_output.interface_revision = 1;
```

非法模式立即报错。price使用NaN表示没有有效价格，而不是价格为零；它当前不进入目标函数。source会成为mock_fixed或mock_periodic，用于追踪数据来源。interface_revision=1标记第1版接口，但当前下游还没有检查版本兼容性。

步骤三 simulate_basic_mpc.m：看完整闭环

这是整套模型的“导演”。它不负责推导优化矩阵，而是安排每个小时先获取窗口、再调用MPC、再调用Decision、最后更新真实水位。

3.1 函数输入与默认值

```
function results = simulate_basic_mpc(config, scenario_mode, n_hours, make_plot)
...
scenario_mode = 'periodic';
n_hours = 72;
make_plot = true;
```

config是参数；scenario_mode选择fixed或periodic；n_hours是闭环仿真时长；make_plot决定是否绘图。默认运行72小时周期场景并绘图。

3.2 为什么生成 n_hours+N-1 个输入点

```
N = config.horizon_steps;
all_data = make_mock_decision_data(datetime(2026, 2, 1), ...
    n_hours + N - 1, scenario_mode);
```

仿真最后一个小时仍然需要一个完整24步窗口。若n_hours=72、N=24，最后循环k=72时要访问72:95，因此总共需要95个数据点，即72+24-1。少了后面23点，最后几个小时就无法组成完整预测窗口。

3.3 预分配和初始状态

```
level = zeros(n_hours + 1, 1);
u_request = zeros(n_hours, 1);
u_actual = zeros(n_hours, 1);
...
level(1) = config.level_initial_m;
u_actual_previous = 0;
```

n_hours个控制动作会产生n_hours+1个水位：初始水位一个，每执行一次再产生一个。预分配zeros提高效率，也让数组尺寸提前明确。上一实际输入初始设为0，用于第一步变化约束。

3.4 每小时截取24步窗口

```
for k = 1:n_hours
    index = k:k+N-1;
    window.demand = all_data.demand(index);
    window.level_ref = all_data.level_ref(index);
    window.time = all_data.time(index);
    window.price = all_data.price(index);
```

第1小时窗口是1:24，第2小时窗口是2:25。窗口每次向前移动一步，这就是滚动时域中的“滚动”。price虽然跟着窗口传递，但基础控制器没有使用它。

3.5 调用MPC

```
solution = constrained_tank_mpc(level(k), window, ...
    config, u_actual_previous);
```

四个输入分别是当前真实水位、未来数据窗口、系统参数、上一小时实际控制。使用上一实际控制而不是上一要求值很重要，因为变化约束应以系统真正执行的状态为起点。

3.6 只执行第一步并经过Decision

```
request = mpc_to_decision_request(solution, all_data.time(k));
decision_result = mock_decision_optimizer(request, config);
```

constrained_tank_mpc虽然算出24步，但request对当前执行最关键的是第一步u_request和requested_flow_m3s。mock Decision当前原样实现这个要求。未来真实模块会在这里把连续流量映射成泵组状态和运行时间。

3.7 用实际流量更新水位

```
level(k+1) = level(k) ...  
    + config.dt_seconds/config.tank_area_m2 ...  
    * (actual_flow_m3s(k) - demand(k));  
u_actual_previous = u_actual(k);
```

这里是闭环物理更新。必须使用actual_flow_m3s，而不是requested_flow_m3s。当前两者相等只是mock执行器的假设；真实Decision接入后两者可能不同。

3.8 保存结果和三幅图

结果结构体保存时间、水位、要求输入、实际输入、要求/实际流量、跟踪误差、需求、参考值和求解器状态。绘图依次显示水位及边界、MPC要求与Decision实际值、用水需求。

results.pump=u_actual只是兼容旧代码的字段名；它表示归一化实际总流量，并不等于某台泵的0/1开关状态。

步骤四 constrained_tank_mpc.m：拆开优化器

这是核心数学文件。它把当前状态和24小时输入转换成标准二次规划，调用quadprog，然后只返回计划的第一步作为当前要求。

4.1 四个输入

function result = constrained_tank_mpc(x_current, decision_output, config, u_previous)	
输入	作用
x_current	当前实测/仿真水位
decision_output	未来需求、参考水位、时间等窗口
config	物理参数、权重和约束
u_previous	上一小时实际归一化流量

如果没有提供u_previous，代码设为0；随后检查当前水位和上一输入是否为有限标量。

4.2 求解器检查与输入整理

```
if exist('quadprog', 'file') ~= 2
    error(...);
end
N = config.horizon_steps;
mpc_input = decision_to_mpc_input(decision_output, N);
demand = mpc_input.demand;
level_ref = mpc_input.level_ref;
```

没有quadprog就停止，因为当前没有备用求解器。接口适配器保证demand和level_ref都是长度N的列向量，再进入数学计算。

4.3 构造预测矩阵

```
dt_over_area = config.dt_seconds / config.tank_area_m2;
lower_sum = tril(ones(N));
Phi = dt_over_area * config.max_requested_flow_m3s * lower_sum;
x_base = x_current*ones(N, 1) ...
    - dt_over_area*lower_sum*demand;
```

lower_sum=L是下三角全1矩阵，负责完成累计。第一行只累积第一个输入，第二行累积前两个，依此类推。

$$X = x_current \cdot 1 - (\Delta t/A)L \cdot demand + (\Delta t/A)q_max L \cdot U$$

代码把前两项命名为x_base，把输入系数命名为Phi，因此 $X = x_base + \Phi \cdot U$ 。

4.4 构造输入差分

```
move_matrix = eye(N);
for row = 2:N
    move_matrix(row, row-1) = -1;
end
previous_vector = zeros(N, 1);
previous_vector(1) = u_previous;
```

move_matrix=D。DU得到 $[u, u-u, u-u, \dots]$ ，再减previous_vector，第一项就变成 $u-u_previous$ 。

$$\Delta U = D U - p$$

4.5 原始目标函数

```
Q = config.weight_level * eye(N);
R = config.weight_input * eye(N);
Rd = config.weight_move * eye(N);
```

$$J = (X - R_ref)' Q (X - R_ref) + U' R U + (DU - p)' Rd (DU - p)$$

这里的 R_{ref} 指参考水位向量，不要与代码中的输入权重矩阵 R 混淆。 Q 、 R 、 R_d 当前都是对角阵，表示每个预测步使用相同权重，步与步之间没有交叉权重。

4.6 展开成 H 和 f

```
H = 2*(Phi'*Q*Phi + R + move_matrix'*Rd*move_matrix);
H = (H + H')/2 + 1e-10*eye(N);
f = 2*(Phi'*Q*(x_base - level_ref) ...
    - move_matrix'*Rd*previous_vector);
```

把 $X=x_{base}+Phi U$ 、 $\Delta U=DU-p$ 代入 J 并展开，所有与 U 无关的常数项可以删掉。quadprog使用 $\frac{1}{2}U^T H U + f^T U$ ，因此二次项前乘2。

$(H+H^T)/2$ 消除浮点计算产生的微小不对称；加 $1e-10I$ 是极小正则化，提高数值稳定性，通常不会实质改变控制策略。

4.7 把水位约束写成 $AU \leq b$

```
A_level = [Phi; -Phi];
b_level = [level_max*ones(N,1)-x_base; ...
    x_base-level_min*ones(N,1)];
```

上界 $X \leq x_{max}$ 代入 $X=x_{base}+Phi U$ ，得到 $Phi U \leq x_{max}-x_{base}$ 。下界 $X \geq x_{min}$ 等价于 $-Phi U \leq x_{base}-x_{min}$ 。上下拼接后一次交给求解器。

4.8 输入变化约束

```
A_move = [move_matrix; -move_matrix];
b_move = [du_max*ones(N,1)+previous_vector; ...
    du_max*ones(N,1)-previous_vector];
```

$|DU-p| \leq du_{max}$ 被拆成两组线性不等式： $DU \leq du_{max}+p$ ，以及 $-DU \leq du_{max}-p$ 。

4.9 输入上下界与求解

```
lb = config.u_min * ones(N, 1);
ub = config.u_max * ones(N, 1);
[u_plan, objective, exitflag, solver_output] = quadprog( ...
    H, f, [A_level; A_move], [b_level; b_move], ...
    [], [], lb, ub, [], options);
```

空的 $[]$ 表示没有等式约束 $Aeq U = beq$ ，也没有指定初始猜测。 u_{plan} 是24步最优计划； $objective$ 是目标函数值； $exitflag$ 说明求解是否成功； $solver_output$ 保存求解器信息。

4.10 失败处理与输出

```
if exitflag <= 0
    error(...);
end
x_prediction = x_base + Phi*u_plan;
result.u_request = u_plan(1);
result.requested_flow_m3s = q_max * result.u_request;
```

$exitflag \leq 0$ 时直接报错，当前没有松弛变量或备用策略，因此不可行问题会中断仿真。成功后重新计算预测水位，并只把 $u_{plan}(1)$ 作为当前执行要求。完整24步计划仍保存在 $u_{request_plan}$ 中，便于检查。

步骤五 三个接口文件：连接MPC与Decision

接口文件的价值不是复杂数学，而是把模块责任分清。如果未来Decision或预测模块更换，只要遵守接口字段，就不必改动MPC核心。

5.1 decision_to_mpc_input.m：把上游数据整理成MPC格式

```
mpc_input.demand = expand_signal(decision_output.demand, ...
    horizon_steps, 'demand');
...
mpc_input.level_ref = expand_signal(decision_output.level_ref, ...
    horizon_steps, 'level_ref');
```

它先检查输入必须是结构体且必须包含demand，再把demand和level_ref整理为长度N的列向量。需求还必须是有有限非负数。

expand_signal的三种情况

输入情况	处理
只有一个标量	复制N次，例如0.025变成24个0.025
长度小于N	报错，因为预测窗口不完整
长度大于或等于N	只取前N个样本

time缺失时用0:N-1代替；price缺失时填NaN；source缺失时写“未说明来源”。注意：这个函数只整理数据，不选择泵，也不运行优化。

5.2 mpc_to_decision_request.m：把MPC结果打包给Decision

```
request.time = request_time;
request.u_request = mpc_result.u_request;
request.requested_flow_m3s = mpc_result.requested_flow_m3s;
request.requested_flow_plan_m3s = ...
    mpc_result.requested_flow_plan_m3s;
request.interface_revision = 1;
```

它先检查当前要求u_request和requested_flow_m3s存在、有限且流量非负，然后打包当前时刻、当前要求以及完整流量计划。Decision最少需要当前要求，完整计划则可用于更复杂的泵组调度。

5.3 mock_decision_optimizer.m：理想执行器

```
u_actual = min(max(request.u_request, config.u_min), config.u_max);
actual_flow_m3s = config.max_requested_flow_m3s * u_actual;
```

min(max())把要求值夹在[u_min,u_max]范围内，这叫饱和处理。因为MPC本身已有相同上下界，正常情况下这里不会改变值；它是一层接口保护。

```
decision_result.tracking_error_m3s = ...
    actual_flow_m3s - request.requested_flow_m3s;
decision_result.mode = '理想连续执行器';
decision_result.pump_status = [];
decision_result.power_kw = NaN;
decision_result.run_time_hours = NaN;
```

跟踪误差定义为实际流量减要求流量。当前理想执行器下应为0。pump_status、power_kw、run_time_hours都是未来真实Decision模块需要填写的字段；空值和NaN明确表示当前没有这些结果。

5.4 接口的整体方向

未来需求/参考值 → decision_to_mpc_input → MPC → mpc_to_decision_request →
Decision → actual_flow → 水箱

名称decision_to_mpc_input容易让人误成“Decision已经决定泵状态”。在当前结构里，它实际承载的是MPC上游数据窗口。真正的执行选择发生在mock_decision_optimizer将来要替换的位置。

把七个实际文件串成一条线

顺序	发生的事情	文件
1	读取物理参数、约束和权重	basic_mpc_config.m
2	生成未来需求和目标水位	make_mock_decision_data.m
3	截取当前24小时窗口	simulate_basic_mpc.m
4	检查并规范输入	decision_to_mpc_input.m
5	建立QP并求出24步计划	constrained_tank_mpc.m
6	把第一步要求打包给Decision	mpc_to_decision_request.m
7	模拟实际执行结果	mock_decision_optimizer.m
8	用实际流量更新水位并滚动	simulate_basic_mpc.m

config → mock data → 24步窗口 → QP计划 → 第一步要求 → 实际执行 → 水位更新 → 下一小时

三个最容易混淆的点

1. 计划流量不等于实际流量

u_request是MPC希望得到的连续流量比例；u_actual是Decision最终实现的比例。当前两者相等，只因为mock执行器是理想的。

2. 24步计划不等于执行24步

MPC每小时求24步，但只执行第一步。完整计划用于看远处，闭环反馈负责修正近处。

3. 基础MPC不等于EMPC

当前目标函数中的u²不是电费。只有把电价、功率与时间正确组成经济成本并进入优化，才开始具备EMPC的核心经济目标。

建议的复习检查题

1. 为什么水箱更新要使用`actual_flow`，而不能直接使用`requested_flow`？
2. 为什么72小时闭环、24步预测需要生成95个输入数据点？
3. 固定需求 $0.025 \text{ m}^3/\text{s}$ 时，为什么稳态输入约为 $u=0.5$ ？
4. `du_max=0.35`限制的是流量本身，还是相邻流量变化？
5. 为什么参考水位是软目标，而 1.40 m 和 3.37 m 是硬边界？
6. `lower_sum`下三角矩阵为什么能表示状态的累计变化？
7. `quadprog`里的`H`、`f`、`A`、`b`分别来自目标函数还是约束？
8. 为什么`price=NaN`不同于`price=0`？
9. 当前`mock Decision`与未来真实`Decision`的关键差别是什么？
10. 为什么这版不能直接称为完整EMPC？

如果这些问题都能不用看代码回答，再回到`constrained_tank_mpc.m`逐行阅读，矩阵就不再是突然出现的符号，而是状态预测、输入差分、目标函数和约束的压缩写法。